Each team member should run **their own service on their own machine**

👉 That's real distribution:

- Different machines
- Network communication over IP
- Independent failures possible

🧠 Final Architecture (Real Distributed Setup)

Instead of:

All services on 1 laptop ❌

You do:

Member 1 (Laptop A) → API Gateway
Member 2 (Laptop B) → Order Service
Member 3 (Laptop C) → Inventory Service
Member 4 (Laptop D) → Payment Worker + Redis
Member 5 (Laptop E) → Frontend

🔗 How They Communicate (IMPORTANT)

Use IP Addresses (not localhost)

Example:

// API Gateway calling Order Service
http://192.168.1.102:5001/create-order

⚙️ What Each Member Must Do

👨‍💻 Member 1 (API Gateway)

- Expose public API
- Routes to:
    - Order Service → http://IP:PORT
    - Inventory → http://IP:PORT

👨‍💻 Member 2 (Order Service)

- Runs on own system
- Publishes event to Redis (remote)

👨‍💻 Member 3 (Inventory Service)

- Runs independently
- Exposes /deduct-stock

👨‍💻 Member 4 (Redis + Payment Worker)

🔥 MOST IMPORTANT NODE

- Redis must be **accessible to all machines**
- Worker consumes queue

👨‍💻 Member 5 (Frontend)

- Calls API Gateway (public IP)

🌐 How to Connect Everyone (2 Options)

☑ Option 1: Same WiFi (EASIEST)

- All laptops on same network
- Use local IPs:

```
ip a   # Linux
ipconfig  # Windows
```

Example:

192.168.1.101 → Gateway
192.168.1.102 → Order
192.168.1.103 → Inventory

🔑 Critical Config Changes

❌ Before (Wrong)

```
ORDER_SERVICE_URL=http://localhost:5001
```

☑ After (Correct)

```
ORDER_SERVICE_URL=http://192.168.1.102:5001
```

📡 Redis Setup (VERY IMPORTANT)

👉 Redis must be centralized

Option:

Run Redis on Member 4 system:

redis-server --bind 0.0.0.0

Then others connect:

REDIS_HOST=192.168.1.104
REDIS_PORT=6379

🧷 Demo You Should Show (Professor Killer 💀)

Scenario:

- Stock = 1
- Two users order simultaneously

Setup:

- Orders from 2 different machines

Result:

- One SUCCESS
- One OUT_OF_STOCK

👉 This proves:

- Distribution ✅
- Concurrency handling ✅
- Atomicity ✅

⚠️ Common Mistakes (Avoid These)

❌ Using localhost everywhere
❌ Running everything on one system
❌ Not exposing ports
❌ Redis not accessible externally

🧠 Important Insight (This Will Impress Prof)

Say this:

👉 “Each microservice is deployed on independent nodes and communicates over network using REST and Redis-based messaging, simulating real distributed system behavior including network latency and partial failures.”

• Repo Setup:

Perfect — now we’ll design this **specifically for a *truly distributed* setup** (each member runs their own service), not just a local monorepo.

I’ll give you:

1. ✅ **Two valid GitHub approaches (choose one)**
2. 📁 **Exact folder structure per service**
3. 🔗 **How all repos connect**
4. ⚠️ **Best approach for your project (important decision)**

🚀 First Decision (VERY IMPORTANT)

❌ Option A: Single Monorepo

- One repo for all services
- Easy, but **not truly distributed ownership**

☑️ Option B: Multi-Repo (RECOMMENDED 🔥)

- Each service = separate GitHub repo
- Each member owns their repo
- Matches **real distributed systems**

👉 For your project → **GO WITH MULTI-REPO**

🧱 Final GitHub Structure (Multi-Repo)

Create **5 repositories:**

ecommerce-api-gateway
ecommerce-order-service
ecommerce-inventory-service
ecommerce-payment-worker
ecommerce-frontend

Optional:

ecommerce-docs (for architecture, diagrams, README)

👥 Repo Ownership (Assign Clearly)

Member	Repo
Member 1	api-gateway
Member 2	order-service
Member 3	inventory-service
Member 4	payment-worker
Member 5	frontend

👉 This ensures:

- True ownership
- Independent deployment
- Real distributed system feel

🗂 Folder Structure (Per Repo)

🧑‍💻 1. API Gateway Repo

```
ecommerce-api-gateway/
│
├── src/
│   ├── routes/
│   │   └── gatewayRoutes.js
│   ├── middleware/
│   │   ├── authMiddleware.js
│   │   └── rateLimiter.js
│   ├── config/
│   │   └── serviceUrls.js   👈 stores IPs of services
│   └── app.js
│
├── package.json
├── Dockerfile
```

```
├── .env
└── README.md
```

📦 2. Order Service Repo

```
ecommerce-order-service/
│
├── src/
│   ├── controllers/
│   │   └── orderController.js
│   ├── routes/
│   │   └── orderRoutes.js
│   ├── services/
│   │   └── orderService.js
│   ├── models/
│   │   └── orderModel.js
│   ├── queue/
│   │   └── producer.js   👈 sends to Redis
│   └── app.js
│
├── package.json
├── Dockerfile
├── .env
└── README.md
```

📦 3. Inventory Service Repo

```
ecommerce-inventory-service/
│
├── src/
│   ├── controllers/
│   │   └── inventoryController.js
│   ├── routes/
│   │   └── inventoryRoutes.js
```

```
│   ├── services/
│   │   └── inventoryService.js
│   ├── db/
│   │   └── queries.sql   👈 atomic SQL here
│   └── app.js
│
├── package.json
├── Dockerfile
├── .env
└── README.md
```

⚙️ 4. Payment Worker Repo

```
ecommerce-payment-worker/
│
├── src/
│   ├── queue/
│   │   ├── worker.js    👈 main worker
│   │   └── queue.js
│   ├── utils/
│   │   └── retryHandler.js
│   └── index.js
│
├── package.json
├── Dockerfile
├── .env
└── README.md
```

🎨 5. Frontend Repo

```
ecommerce-frontend/
│
├── src/
│   ├── components/
```

```
│   ├── pages/
│   ├── services/
│   │   └── api.js   👈 calls API Gateway
│   └── App.js
│
├── package.json
├── Dockerfile
└── README.md
```

🔗 How These Repos Connect (IMPORTANT)

Since repos are separate:

👉 Connection happens via **environment variables**

Example: API Gateway .env

```
ORDER_SERVICE_URL=http://192.168.1.102:5001
INVENTORY_SERVICE_URL=http://192.168.1.103:5002
```

Example: Order Service .env

```
REDIS_HOST=192.168.1.104
REDIS_PORT=6379
```

Example: Frontend .env

```
API_GATEWAY_URL=http://192.168.1.101:5000
```

📄 README Template (Each Repo Must Have)

```
# Service Name

## Description
What this service does

## Tech Stack
Node.js, Express, MySQL, Redis

## Setup
npm install
npm start

## Environment Variables
(Add .env details)

## API Endpoints
- /create-order
- /deduct-stock
```

🔥 Super Important (This Will Impress Prof)

Create **one central documentation repo**:

ecommerce-docs/

Inside:

```
├── architecture.png
├── system-design.md
```

├── service-communication.md
└── demo-scenarios.md

👉 Put:

- Architecture diagram
- Flow explanation
- Distributed setup explanation

⚠ Critical Advice (Listen Carefully)

👉 Multi-repo is harder but:

- Looks **industry-level**
- Shows **true distribution**
- Gives each member **ownership**

👉 But risk:

- Integration issues

✓ Solution:

- Fix IPs early
- Test daily